Practical 1

Multiple Inheritance using Interfaces

Aim: Write a C# program to demonstrate Multiple Inheritance using Interfaces.

Project / namespace (as in source): Not stated in source (no namespace in code)

Procedure:

1. Open Visual Studio and click Create a new project.
2. In the template search box type Console App, select the C# Console App template and click Next.
3. Enter the Project name (a name of your choice), choose the Location folder and click Next.
4. On the Additional information screen select the framework version installed on your system and click Create.
5. In Solution Explorer double-click Program.cs, delete the existing template code and enter the source code given below.
6. Press Ctrl+S to save, then choose Build > Build Solution (Ctrl+Shift+B). The Error List must show 0 errors.
7. Choose Debug > Start Without Debugging (Ctrl+F5) to run the program. A console window opens.
8. Observe the two lines printed by d.Print() and d.Show(); both interface methods are implemented by the single class Demo.
9. Press Enter to close the console window, and take a screenshot of the output for the Output section.

Source Code:

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;

interface IPrint
{
    void Print();
}

interface IShow
{
    void Show();
}

class Demo : IPrint, IShow
{
    public void Print()
    {
        Console.WriteLine("Print method called");
    }
```

```
    public void Show()
    {
        Console.WriteLine("Show method called");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Demo d = new Demo();
        d.Print();
        d.Show();
        Console.ReadLine();
    }
}
```

Output:

Expected output (derived by reading the code; not captured from a run):

```
Print method called
Show method called
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 2

Constructor Overloading

Aim: Write a C# program to demonstrate constructor overloading by defining multiple constructors with different parameter list in the same class.

Project / namespace (as in source): Pract2cContructorOverlaoding

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. In the template search box type Console App, select the C# Console App template and click Next.
3. Enter the Project name (Pract2cContructorOverlaoding), choose the Location folder and click Next.
4. On the Additional information screen select the framework version installed on your system and click Create.
5. In Solution Explorer double-click Program.cs, delete the existing template code and enter the source code given below.
6. Press Ctrl+S to save, then choose Build > Build Solution (Ctrl+Shift+B). The Error List must show 0 errors.
7. Choose Debug > Start Without Debugging (Ctrl+F5) to run the program. A console window opens.
8. Observe the output: each of the three objects (s1, s2, s3) calls a different constructor, selected by the number and type of arguments.
9. Press Enter to close the console window, and take a screenshot of the output for the Output section.

Source Code:

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Pract2cContructorOverlaoding
{
    class Students
    {
        //Default Constructor
        public Students()
        {
            Console.WriteLine("Default Constructor called");
        }
```

```
        //Parameterized Constructor
        public Students(string name)
        {
            Console.WriteLine("Student name:" + name);
        }

        //Another Parameterized Constructor
        public Students(string name, int age)
        {
            Console.WriteLine("Student name:" + name);
            Console.WriteLine("Student Age:" + age);
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Students s1 = new Students();
            Students s2 = new Students("Ayesha");
            Students s3 = new Students("Ayesha", 20);

            Console.ReadLine();
        }
    }
}
```

Output:

Expected output (derived by reading the code; not captured from a run):

```
Default Constructor called
Student name:Ayesha
Student name:Ayesha
Student Age:20
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 3

Interfaces

Aim: Write a C# program to demonstrate the use of interfaces by defining contracts that classes must implement.

Project / namespace (as in source): P3Interface

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. In the template search box type Console App, select the C# Console App template and click Next.
3. Enter the Project name (P3Interface), choose the Location folder and click Next.
4. On the Additional information screen select the framework version installed on your system and click Create.
5. In Solution Explorer double-click Program.cs, delete the existing template code and enter the source code given below.
6. Press Ctrl+S to save, then choose Build > Build Solution (Ctrl+Shift+B). The Error List must show 0 errors.
7. Choose Debug > Start Without Debugging (Ctrl+F5) to run the program. A console window opens.
8. Observe the output: the Sqaure class implements the Area() method declared in the IShape interface and prints the area of a square with side 5.
9. Press Enter to close the console window, and take a screenshot of the output for the Output section.

Source Code:

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace P3Interface

{
    interface IShape
    {
        void Area();
    }

    class Sqaure : IShape
    {
```

```
        int side = 5;

        public void Area()
        {
            Console.WriteLine("Area of Square= " + (side * side));
        }
    }

    internal class Program
    {
        static void Main(string[] args)
        {
            Sqaure sqaure = new Sqaure();
            sqaure.Area();
            Console.ReadLine();
        }
    }
}
```

Output:

Expected output (derived by reading the code; not captured from a run):

```
Area of Square= 25
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 4

Exception Handling with Custom Exception and Multiple Catch

Aim: Write a C# program to demonstrate exception handling using try-catch-finally blocks, custom exceptions, and multiple catch handlers.

Project / namespace (as in source): Not stated in source (no namespace in code)

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. In the template search box type Console App, select the C# Console App template and click Next.
3. Enter the Project name (a name of your choice), choose the Location folder and click Next.
4. On the Additional information screen select the framework version installed on your system and click Create.
5. In Solution Explorer double-click Program.cs, delete the existing template code and enter the source code given below.
6. Press Ctrl+S to save, then choose Build > Build Solution (Ctrl+Shift+B). The Error List must show 0 errors.
7. Choose Debug > Start Without Debugging (Ctrl+F5) to run the program. A console window opens.
8. Run the program four times, entering 1, -1, 5 and abc in turn when 'Enter age:' is shown, and note which catch block handles each input; the finally block runs every time.
9. Press Enter to close the console window, and take a screenshot of the output for the Output section.

Source Code:

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;

class InvalidAgeException : Exception
{
    public InvalidAgeException(string message) : base(message)
    {
    }
}

class Program
{
    static void Main(string[] args)
    {
        try
        {
```

```
            Console.Write("Enter age: ");
            int age = Convert.ToInt32(Console.ReadLine());

            if (age < 0)
                throw new InvalidAgeException("Age cannot be negative.");

            int[] numbers = { 10, 20, 30 };
            Console.WriteLine("Selected value: " + numbers[age]);
        }
        catch (InvalidAgeException ex)
        {
            Console.WriteLine("Custom Exception: " + ex.Message);
        }
        catch (IndexOutOfRangeException ex)
        {
            Console.WriteLine("Index error: " + ex.Message);
        }
        catch (FormatException)
        {
            Console.WriteLine("Invalid input.");
        }
        catch (Exception ex)
        {
            Console.WriteLine("Exception: " + ex.Message);
        }
        finally
        {
            Console.WriteLine("Program Ended");
        }

        Console.ReadLine();
    }
}
```

Output:

Expected output for sample inputs (derived by reading the code; not captured from a run):

```
Enter age: 1
Selected value: 20
Program Ended

Enter age: -1
Custom Exception: Age cannot be negative.
Program Ended

Enter age: 5
Index error: Index was outside the bounds of the array.
Program Ended

Enter age: abc
Invalid input.
Program Ended
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 5

Pascal's Triangle using Nested Loops

Aim: Write a C# program to accept the number of rows from the user and display Pascal's triangle using nested loops.

Project / namespace (as in source): Not stated in source (no namespace in code)

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. In the template search box type Console App, select the C# Console App template and click Next.
3. Enter the Project name (a name of your choice), choose the Location folder and click Next.
4. On the Additional information screen select the framework version installed on your system and click Create.
5. In Solution Explorer double-click Program.cs, delete the existing template code and enter the source code given below.
6. Press Ctrl+S to save, then choose Build > Build Solution (Ctrl+Shift+B). The Error List must show 0 errors.
7. Choose Debug > Start Without Debugging (Ctrl+F5) to run the program. A console window opens.
8. When 'Enter number of rows:' appears, type a number such as 5 and press Enter; Pascal's triangle with that many rows is printed.
9. Press Enter to close the console window, and take a screenshot of the output for the Output section.

Source Code:

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;

class Program
{
    static void Main(string[] args)
    {
        Console.Write("Enter number of rows: ");
        int n = Convert.ToInt32(Console.ReadLine());

        for (int i = 0; i < n; i++)
        {
            int number = 1;

            for (int j = 0; j <= i; j++)
```

```
            {
                Console.Write(number + " ");
                number = number * (i - j) / (j + 1);
            }

            Console.WriteLine();
        }

        Console.ReadLine();
    }
}
```

Output:

Expected output for 5 rows (computed from the code's formula; not captured from a run):

```
Enter number of rows: 5
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 6

PWA with a Custom Offline Page

Aim: Create a PWA that displays a custom offline page whenever the user loses internet connectivity. Use a Service Worker to cache the required resources.

Project / namespace (as in source): Not stated in source (ASP.NET Core MVC; template-style Program.cs)

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. Select ASP.NET Core Web App (Model-View-Controller) and click Next.
3. Enter the Project name and Location, click Next, choose the installed .NET framework, keep Authentication type as None and Configure for HTTPS ticked, then click Create.
4. In Solution Explorer expand the project and locate the wwwroot folder (it already contains css, js and lib).
5. Right-click wwwroot > Add > New Item, choose HTML Page, name it offline.html and enter the code given below.
6. Right-click wwwroot > Add > New Item, choose JavaScript File, name it service-worker.js and enter the code given below.
7. Open the existing wwwroot/js/site.js and enter the Service Worker registration code given below (keep any code already present in the file).
8. Right-click wwwroot > Add > New Item, choose a JSON File (or Text File), name it manifest.json and enter the manifest code given below.
9. Open Views/Shared/_Layout.cshtml, confirm that site.js is loaded with a script tag (the MVC template includes it by default) and add the manifest link given below inside the <head> section.
10. Open Program.cs and confirm that app.UseStaticFiles() is present, so that the files under wwwroot are served.
11. Check that every path in the cache list of service-worker.js exists in the project: /, /Home/Index, /css/site.css, /js/site.js and /offline.html. A missing resource makes the cache installation fail.
12. Save all files (Ctrl+S) and run the project with Ctrl+F5. Open the website once while connected to the internet so that the Service Worker can install and cache the files.
13. Press F12 to open Developer Tools. Under Application > Service Workers confirm that service-worker.js is activated, and under Cache Storage confirm that student-record-pwa-v1 lists the cached files.
14. Open the Network tab and select Offline in the throttling drop-down.
15. Refresh the page, or navigate to a page that is not cached, and check that the custom "You are Offline" page appears.

16. Untick Offline to restore the connection and take screenshots of the online page, the Service Worker / Cache Storage panel and the offline page.

Source Code:

wwwroot/offline.html

Source: your uploaded practicals document - copied exactly (not modified).

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>You are Offline</title>
</head>
<body>
    <div class="offline-box">
        <div class="icon">📡</div>
        <h1>You are Offline</h1>
        <p>The College Information Portal is currently unable to connect to the
Internet.</p>
        <p>Please check your Internet connection and try again.</p>
        <button onclick="location.reload()">Try Again</button>
    </div>
</body>
</html>
```

wwwroot/service-worker.js

Source: your uploaded practicals document - copied exactly (not modified).

```
const CACHE_NAME = "student-record-pwa-v1";

const urlsToCache = [
    "/",
    "/Home/Index",
    "/css/site.css",
    "/js/site.js",
    "/offline.html"
];

// INSTALL EVENT
self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );
    self.skipWaiting();
});

// ACTIVATE EVENT
self.addEventListener("activate", event => {
    event.waitUntil(
        caches.keys()
            .then(cacheNames => {
                return Promise.all(
                    cacheNames
                        .filter(name => name !== CACHE_NAME)
```

```
                        .map(name => caches.delete(name))
                );
            })
    );
    self.clients.claim();
});

// FETCH EVENT
self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request)
            .then(response => {
                return response;
            })
            .catch(() => {
                return caches.match(event.request)
                    .then(response => {
                        if (response) {
                            return response;
                        }
                        return caches.match("/offline.html");
                    });
            })
    );
});
```

wwwroot/js/site.js

Source: your uploaded practicals document - copied exactly (not modified).

```
// Please see documentation at https://learn.microsoft.com/aspnet/core/client-
side/bundling-and-minification
// for details on configuring this project to bundle and minify static web assets.

// Write your JavaScript code.

if ("serviceWorker" in navigator) {
    window.addEventListener("load", () => {
        navigator.serviceWorker
            .register("/service-worker.js")
            .then(registration => {
                console.log("Service Worker registered:", registration.scope);
            })
            .catch(error => {
                console.log("Service Worker registration failed:", error);
            });
    });
}
```

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();
```

```
// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    // The default HSTS value is 30 days. You may want to change this for
production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();

app.UseAuthorization();

app.MapStaticAssets();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();


app.Run();
```

wwwroot/manifest.json

Source: the file you attached later - copied exactly. NOT in your practicals document (that document has no manifest for Practical 6).

```
{
    "name": "Offline PWA",
    "short_name": "OfflinePWA",
    "start_url": "/",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#102a43"
}
```

Views/Shared/_Layout.cshtml - add inside <head>

Source: the file you attached later - copied exactly. NOT in your practicals document.

```
<link rel="manifest" href="/manifest.json">
```

Output:

Expected behaviour (derived by reading the code; not tested in this audit):

While online, pages load normally and the Service Worker caches /, /Home/Index, /css/site.css, /js/site.js and /offline.html. When the network is unavailable, a request that is not in the cache is answered with /offline.html ("You are Offline" with a Try Again button).

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 7

MVC Application for Basic Arithmetic Operations

Aim: MVC Application for Basic Arithmetic Operations.

Project / namespace (as in source): Model namespace P7ArithmeticMvc.Models; controller namespace Pract1aArithmeticMVC.Controllers (both as in source)

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. Search for ASP.NET Core Web App (Model-View-Controller), select it and click Next.
3. Enter the Project name (the model namespace in your source is P7ArithmeticMvc), choose the Location, click Next, select the installed .NET framework and click Create.
4. In Solution Explorer right-click the Models folder > Add > Class, name it Calculator.cs and enter the Model code given below.
5. Open Controllers/HomeController.cs and replace its content with the Controller code given below. It has a GET Index() action that shows the form and a POST Index(Calculator calc) action that performs the calculation.
6. Open Views/Home/Index.cshtml and replace its content with the View code given below. It contains the form with two number boxes, an operator drop-down and a Calculate button.
7. Save all files (Ctrl+S) and run the project with Ctrl+F5. The browser opens the Arithmetic Operations page.
8. Enter Number 1 (for example 10) and Number 2 (for example 5), select an operation from the drop-down and click Calculate.
9. The result is displayed below the form as Result = value. Repeat for Subtraction, Multiplication and Division and take a screenshot of each output.

Source Code:

Models/Calculator.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
namespace P7ArithmeticMvc.Models
{
    public class Calculator
    {
        public double Number1 { get; set; }
        public double Number2 { get; set; }
        public required String Operation { get; set; }
        public double Result { get; set; }
    }
}
```

Controllers/HomeController.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using Microsoft.AspNetCore.Mvc;
using P7ArithmeticMvc.Models;
using System.Diagnostics;

namespace Pract1aArithmeticMVC.Controllers
{
    public class HomeController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        public IActionResult Privacy()
        {
            return View();
        }

        [HttpPost]
        public IActionResult Index(Calculator calc)
        {
            switch (calc.Operation)
            {
                case "+":
                    calc.Result = calc.Number1 + calc.Number2;
                    break;

                case "-":
                    calc.Result = calc.Number1 - calc.Number2;
                    break;

                case "*":
                    calc.Result = calc.Number1 * calc.Number2;
                    break;

                case "/":
                    if (calc.Number2 != 0)
                        calc.Result = calc.Number1 / calc.Number2;
                    break;
            }

            return View(calc);
        }

        public IActionResult Error()
        {
            return View();
        }
    }
}
```

Views/Home/Index.cshtml

Source: your uploaded practicals document - copied exactly (not modified).

```
@model P7ArithmeticMvc.Models.Calculator
```

```
<h2>Arithmetic Operations</h2>

<form action="/" method="post">
    <label asp-for="Number1">Enter Number 1:</label>
    <br />
    <input type="number" asp-for="Number1" />
    <br />
    <br />

    <label>Enter Number 2:</label>
    <br />
    <input type="number" asp-for="Number2" />
    <br />
    <br />

    <select asp-for="Operation">
        <option value="op">Select Operator</option>
        <option value="+">Addition</option>
        <option value="-">Subtraction</option>
        <option value="*">Multiplication</option>
        <option value="/">Division</option>
    </select>
    <br />
    <br />

    <input type="submit" value="Calculate" />
</form>

<br />

@if (Model != null)
{
    <h3>Result = @Model.Result</h3>
}
```

Output:

Expected behaviour (derived by reading the code; not tested in this audit):

The page shows two number boxes, an operator drop-down and a Calculate button. For example, 10 + 5 with Addition shows "Result = 15".

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 8

Floyd's Triangle in C#

Aim: Create an application to print Floyd's triangle till n rows in C#.

Project / namespace (as in source): Pract1bFloydTriangle

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. In the template search box type Console App, select the C# Console App template and click Next.
3. Enter the Project name (Pract1bFloydTriangle), choose the Location folder and click Next.
4. On the Additional information screen select the framework version installed on your system and click Create.
5. In Solution Explorer double-click Program.cs, delete the existing template code and enter the source code given below.
6. Press Ctrl+S to save, then choose Build > Build Solution (Ctrl+Shift+B). The Error List must show 0 errors.
7. Choose Debug > Start Without Debugging (Ctrl+F5) to run the program. A console window opens.
8. When 'Enter number of rows:' appears, type a number such as 5 and press Enter; Floyd's triangle with that many rows is printed.
9. Press Enter to close the console window, and take a screenshot of the output for the Output section.

Source Code:

Program.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;

namespace Pract1bFloydTriangle
{
    internal class Program
    {
        static void Main(string[] args)
        {
            int n, number = 1;

            Console.Write("Enter number of rows:");
            n = Convert.ToInt32(Console.ReadLine());

            for (int i = 1; i <= n; i++)
            {
                for (int j = 1; j <= i; j++)
```

```
                {
                    Console.Write(number + "    ");
                    number++;
                }

                Console.WriteLine();
            }

            Console.ReadLine();
        }
    }
}
```

Output:

Expected output for 5 rows (computed from the code logic; not captured from a run):

```
Enter number of rows:5
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 9

PWA that Displays a Custom Offline Page

Aim: To develop a Progressive Web Application (PWA) that displays a custom offline page when the user loses internet connectivity, using a Service Worker to cache the required resources.

Project / namespace (as in source): OfflinePWA_Q9 (ASP.NET Core Web App - Model-View-Controller), as named in your attached file

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. Select ASP.NET Core Web App (Model-View-Controller) and click Next.
3. Set the Project name to OfflinePWA_Q9, choose the Location, click Next, select the installed .NET version and click Create.
4. In Solution Explorer expand the project and locate the wwwroot folder.
5. Right-click wwwroot > Add > New Item, choose HTML Page, name it offline.html and enter the code given below.
6. Right-click wwwroot > Add > New Item, choose JavaScript File, name it service-worker.js and enter the code given below.
7. Open the existing wwwroot/js/site.js, keep the code already present and append the Service Worker registration code given below.
8. Create wwwroot/manifest.json and enter the manifest code given below (the cache list in service-worker.js includes /manifest.json).
9. Open Views/Shared/_Layout.cshtml (or Views/Home/Index.cshtml) and add the manifest link given below inside the <head> section.
10. Open Program.cs and confirm that the lines given below are present; app.UseStaticFiles() is required so that the files under wwwroot are served.
11. Check that every path in the cache list exists in the project, especially /manifest.json, /css/site.css and /js/site.js. A missing resource makes the cache installation fail.
12. Save all files and run the website with Ctrl+F5. Open the website once while connected to the internet so that the Service Worker can install and cache the files.
13. Press F12 to open Developer Tools, open the Network tab and select Offline.
14. Refresh the page, or navigate to a page that is not cached, and check that the custom "You are Offline" page appears.
15. Untick Offline to restore the connection and take screenshots of each stage.

Source Code:

wwwroot/offline.html

Source: the file you attached later (your Practical 6/9 text) - copied exactly.

```
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>You are Offline</title>
</head>
<body>
    <div class="offline-box">
        <div class="icon">📡</div>
        <h1>You are Offline</h1>
        <p>
            The College Information Portal is currently
            unable to connect to the Internet.
        </p>
        <p>
            Please check your Internet connection
            and try again.
        </p>
        <button onclick="location.reload()">Try Again</button>
    </div>
</body>
</html>
```

wwwroot/service-worker.js

Source: the file you attached later (your Practical 6/9 text) - copied exactly.

```
const CACHE_NAME = "student-record-pwa-v1";

const urlsToCache = [
    "/",
    "/Home/Index",
    "/css/site.css",
    "/js/site.js",
    "/manifest.json",
    "/offline.html"
];

// INSTALL EVENT
self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache => {
                return cache.addAll(urlsToCache);
            })
    );

    self.skipWaiting();
});

// ACTIVATE EVENT
self.addEventListener("activate", event => {
    event.waitUntil(
        caches.keys()
            .then(cacheNames => {
```

```
                return Promise.all(
                    cacheNames
                        .filter(name => name !== CACHE_NAME)
                        .map(name => caches.delete(name))
                );
            })
    );

    self.clients.claim();
});

// FETCH EVENT
self.addEventListener("fetch", event => {
    event.respondWith(
        fetch(event.request)
            .then(response => {
                return response;
            })
            .catch(() => {
                return caches.match(event.request)
                    .then(cachedResponse => {
                        if (cachedResponse) {
                            return cachedResponse;
                        }

                        return caches.match("/offline.html");
                    });
            })
    );
});
```

wwwroot/js/site.js (append to the existing file)

Source: the file you attached later (your Practical 6/9 text) - copied exactly.

```
if ("serviceWorker" in navigator) {
    window.addEventListener("load", () => {
        navigator.serviceWorker
            .register("/service-worker.js")
            .then(registration => {
                console.log(
                    "Service Worker registered:",
                    registration.scope
                );
            })
            .catch(error => {
                console.log(
                    "Service Worker registration failed:",
                    error
                );
            });
    });
}
```

wwwroot/manifest.json

Source: the file you attached later (your Practical 6/9 text) - copied exactly.

```
{
```

```
    "name": "Offline PWA",
    "short_name": "OfflinePWA",
    "start_url": "/",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#102a43"
}
```

Views/Shared/_Layout.cshtml - add inside <head>

Source: the file you attached later (your Practical 6/9 text) - copied exactly.

```
<link rel="manifest" href="/manifest.json">
```

Program.cs (relevant lines only - not a replacement for the whole file)

Source: the file you attached later (your Practical 6/9 text) - copied exactly.

```
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseRouting();
app.UseAuthorization();
```

Output:

Expected output (as given in your attached file):

```
 You are Offline
The College Information Portal is currently unable to connect to the Internet.

Please check your Internet connection and try again.

[ Try Again ]
```

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 10

Cache-First PWA using a Service Worker

Title: Develop a Progressive Web Application (PWA) that supports offline functionality using a Service Worker and the cache-first caching strategy.

Aim: To develop a Progressive Web Application that can load its cached resources and provide basic functionality when the device is offline, using a Service Worker and the cache-first caching strategy.

Project (as in source): OfflinePWA - ASP.NET Core Web App (Model-View-Controller)

Software and Requirements:

- Microsoft Visual Studio
- ASP.NET Core Web App (Model-View-Controller) project template
- A supported .NET SDK installed with Visual Studio
- Microsoft Edge or Google Chrome
- Developer Tools for testing offline behavior

Project Setup:

Create a new ASP.NET Core Web App (Model-View-Controller) project in Visual Studio. Use the project name OfflinePWA. Select Authentication type: None, enable Configure for HTTPS, and leave Docker disabled if the option is shown. Select an installed supported .NET framework version.

The following files are created inside the wwwroot folder so that ASP.NET Core can serve them as static files:

```
OfflinePWA/
├── Controllers/
├── Models/
├── Views/
├── wwwroot/
│   ├── index.html
│   ├── style.css
│   ├── manifest.json
│   ├── app.js
│   └── service-worker.js
└── Program.cs
```

Procedure:

1. Open Visual Studio and choose Create a new project.
2. Search for ASP.NET Core Web App (Model-View-Controller), select it, and click Next.
3. Set Project name and Solution name to OfflinePWA. Choose your preferred coding folder and click Next.
4. On Additional information, select an installed supported .NET framework, choose Authentication type None, enable Configure for HTTPS, and leave Docker disabled if shown. Click Create.
5. In Solution Explorer, expand the project and locate wwwroot.

6. Right-click wwwroot, choose Add > New Item, and create each file listed below. Ensure each filename is spelled exactly as shown.
7. Paste the corresponding code into each file and save it using Ctrl+S.
8. Run the project with Ctrl+F5 or the Visual Studio Run button.
9. Open the local HTTPS address shown by Visual Studio, followed by /index.html, for example https://localhost:7258/index.html. Use the actual port shown by your project.
10. Keep the page open and connected to the internet initially so that the Service Worker can install and cache the required files.

Source Code:

File 1: wwwroot/index.html

Purpose: Defines the webpage content and links the manifest, stylesheet, and JavaScript file.

Source: your practicals document, Practical No. 10 write-up - copied exactly (not modified).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="theme-color" content="#2563eb">
    <title>Offline PWA</title>
    <link rel="manifest" href="manifest.json">
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h1>Progressive Web Application</h1>
        <p>This application supports offline functionality.</p>
        <p id="status">Checking connection...</p>
        <button onclick="showMessage()">Click Me</button>
        <p id="message"></p>
    </div>

    <script src="app.js"></script>
</body>
</html>
```

File 2: wwwroot/style.css

Purpose: Styles the page and its button.

Source: your practicals document, Practical No. 10 write-up - copied exactly (not modified).

```
body {
    font-family: Arial, sans-serif;
    background-color: #f0f4f8;
    text-align: center;
    margin: 0;
    padding: 50px 20px;
}

.container {
    background-color: white;
    max-width: 600px;
```

```
    margin: auto;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
}

h1 {
    color: #2563eb;
}

button {
    background-color: #2563eb;
    color: white;
    border: none;
    padding: 12px 24px;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background-color: #1d4ed8;
}
```

File 3: wwwroot/manifest.json

Purpose: Provides the PWA name, launch URL, display mode, and theme colors.

Source: your practicals document, Practical No. 10 write-up - copied exactly (not modified).

```
{
  "name": "Offline Progressive Web Application",
  "short_name": "Offline PWA",
  "start_url": "./index.html",
  "scope": "./",
  "display": "standalone",
  "background_color": "#f0f4f8",
  "theme_color": "#2563eb"
}
```

File 4: wwwroot/app.js

Purpose: Registers the Service Worker and updates the displayed online/offline status.

Source: your practicals document, Practical No. 10 write-up - copied exactly (not modified).

```
if ("serviceWorker" in navigator) {
    window.addEventListener("load", async () => {
        try {
            await navigator.serviceWorker.register("./service-worker.js");
            console.log("Service Worker registered successfully.");
        } catch (error) {
            console.error("Service Worker registration failed:", error);
        }
    });
}

function updateStatus() {
    const status = document.getElementById("status");
```

```
    if (navigator.onLine) {
        status.textContent = "You are Online";
    } else {
        status.textContent = "You are Offline";
    }
}

function showMessage() {
    document.getElementById("message").textContent =
        "The application is working!";
}

window.addEventListener("online", updateStatus);
window.addEventListener("offline", updateStatus);

updateStatus();
```

File 5: wwwroot/service-worker.js

Purpose: Caches application resources during installation and applies the cache-first strategy to same-origin GET requests.

Source: your practicals document, Practical No. 10 write-up - copied exactly (not modified).

```
const CACHE_NAME = "pwa-cache-v1";

const FILES_TO_CACHE = [
    "./",
    "./index.html",
    "./style.css",
    "./manifest.json",
    "./app.js"
];

// Install the Service Worker and cache application files
self.addEventListener("install", (event) => {
    event.waitUntil(
        caches.open(CACHE_NAME).then((cache) => {
            return cache.addAll(FILES_TO_CACHE);
        })
    );

    self.skipWaiting();
});

// Activate the Service Worker and remove old caches
self.addEventListener("activate", (event) => {
    event.waitUntil(
        caches.keys().then((cacheNames) => {
            return Promise.all(
                cacheNames
                    .filter((name) => name !== CACHE_NAME)
                    .map((name) => caches.delete(name))
            );
        })
    );

    self.clients.claim();
});
```

```
// Cache-first strategy
self.addEventListener("fetch", (event) => {
    if (event.request.method !== "GET") {
        return;
    }

    const url = new URL(event.request.url);

    // Handle only requests from this application
    if (url.origin !== self.location.origin) {
        return;
    }

    event.respondWith(
        caches.match(event.request).then((cachedResponse) => {
            // Return the cached response if available
            if (cachedResponse) {
                return cachedResponse;
            }

            // Otherwise, try the network
            return fetch(event.request).then((networkResponse) => {
                if (networkResponse.ok) {
                    const responseToCache = networkResponse.clone();

                    caches.open(CACHE_NAME).then((cache) => {
                        cache.put(event.request, responseToCache);
                    });
                }

                return networkResponse;
            });
        }).catch(() => {
            // Show the cached homepage for offline navigation
            if (event.request.mode === "navigate") {
                return caches.match("./index.html");
            }

            return new Response("You are offline and this resource is not cached.",
{
                status: 503,
                headers: {
                    "Content-Type": "text/plain"
                }
            });
        })
    );
});
```

Program.cs Check

The MVC template normally includes static-file middleware. Confirm that app.UseStaticFiles() is present before endpoint mapping. To open index.html at the site root, app.UseDefaultFiles() must be placed before app.UseStaticFiles(). Alternatively, open /index.html directly as described above. Do not add duplicate middleware if the required setup already exists.

```
app.UseDefaultFiles();
app.UseStaticFiles();
```

If you use the two lines above, place them after app.UseHttpsRedirection() and before app.UseRouting() or endpoint mapping, following the middleware structure already present in your Program.cs. If using only /index.html, app.UseStaticFiles() is the key requirement.

MISSING: The complete Program.cs for this project is not in your practicals document (only the two-line guidance above).

How to Run the Application:

11. Save all files using Ctrl+S.
12. Run the project using Ctrl+F5 or the Run button in Visual Studio.
13. Copy the HTTPS address shown by Visual Studio and add /index.html. Example: https://localhost:7258/index.html. The port may differ on your computer.
14. Wait for the page to load while online. The Service Worker registers after the page loads.
15. Open Developer Tools with F12 and select the Application tab (in Edge or Chrome).
16. Under Service Workers, check that service-worker.js is registered and activated.
17. Under Cache Storage, check that the pwa-cache-v1 cache contains the application files.
18. Open the Network tab and enable Offline in the network throttling dropdown.
19. Reload the page. Confirm that the cached page still appears and the Click Me button displays its message.
20. Turn Offline off to restore the connection.

Expected Output:

The page displays the heading “Progressive Web Application,” a short offline-functionality message, the current online/offline status, and a Click Me button. Clicking the button displays “The application is working!” After the files have been cached, the page can be loaded while offline.

[Insert your output screenshots here (page online, Cache Storage, page offline) - none were supplied, so none were added]

Important Notes / Troubleshooting:

- The first visit requires a network connection so the Service Worker can install and cache the files.
- Service Workers require HTTPS or localhost. Do not test by opening the HTML file directly using a file:// URL.
- If the Service Worker does not register, confirm that service-worker.js is under wwwroot and that static files are enabled.
- If cache.addAll fails, confirm every path in FILES_TO_CACHE exists and is accessible at the current application path.

- If you change the cached resources, change CACHE_NAME (for example, pwa-cache-v2), save, reload the page, and check the updated cache.
- If your MVC project uses a non-root path or a custom route base, the relative paths and Service Worker scope may need to be adjusted to that base path.

Practical 11

ASP.NET Web Forms AdRotator with Advertisement XML

Aim: Create an ASP.NET Web Forms application that displays rotating advertisements using the AdRotator control and an advertisement XML file.

Project / namespace (as in source): P11StudentRegistration (ASP.NET Web Forms)

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project.
2. Search for ASP.NET Web Application (.NET Framework), select the C# template and click Next.
3. Enter the Project name (P11StudentRegistration in your source), choose the Location and click Create, then select the Web Forms template and click Create.
4. In Solution Explorer right-click the project > Add > New Item, choose Web Form, name it AdRotator.aspx and click Add. Visual Studio also creates AdRotator.aspx.cs.
5. Open AdRotator.aspx and enter the markup given below, containing the asp:AdRotator control with AdvertisementFile="~/Advertisements.xml".
6. Open AdRotator.aspx.cs and confirm the code-behind given below.
7. Right-click the project > Add > New Item, choose XML File, name it Advertisements.xml and enter the advertisement entries given below (ImageUrl, NavigateUrl, AlternateText, Impressions, Keyword).
8. Right-click the project > Add > New Folder, name it Images, then right-click Images > Add > Existing Item and add ad1.jpg, ad2.jpg and ad3.jpg.
9. Right-click AdRotator.aspx > Set As Start Page, then press Ctrl+F5 to run.
10. The browser shows the Advertisement Rotator heading and one advertisement. Press F5 to refresh several times; a different advertisement appears according to the Impressions weights. Clicking an advertisement opens its NavigateUrl.
11. Take screenshots of the page showing different advertisements.

Source Code:

AdRotator.aspx

Source: your uploaded practicals document - copied exactly (not modified).

```
<%@ Page Language="C#" AutoEventWireup="true" CodeBehind="AdRotator.aspx.cs"
Inherits="P11StudentRegistration.AdRotator" %>

<!DOCTYPE html>
<html>
<head runat="server">
    <title>AdRotator Demo</title>
</head>
<body>
    <form id="form1" runat="server">
```

```
        <h2>Advertisement Rotator</h2>

        <asp:AdRotator ID="AdRotator1" runat="server"
            AdvertisementFile="~/Advertisements.xml"
            Width="500px" Height="200px" />
    </form>
</body>
</html>
```

AdRotator.aspx.cs

Source: your uploaded practicals document - copied exactly (not modified).

```
using System;

namespace P11StudentRegistration
{
    public partial class AdRotator : System.Web.UI.Page
    {
        protected void Page_Load(object sender, EventArgs e)
        {
        }
    }
}
```

Advertisements.xml

Source: your uploaded practicals document - copied exactly (not modified).

```
<?xml version="1.0" encoding="utf-8" ?>
<Advertisements>
       <Ad>
              <ImageUrl>~/Images/ad1.jpg</ImageUrl>
              <NavigateUrl>https://www.microsoft.com/</NavigateUrl>
              <AlternateText>Microsoft</AlternateText>
              <Impressions>50</Impressions>
              <Keyword>Technology</Keyword>
       </Ad>
       <Ad>
              <ImageUrl>~/Images/ad2.jpg</ImageUrl>
              <NavigateUrl>https://visualstudio.microsoft.com/</NavigateUrl>
              <AlternateText>Visual Studio</AlternateText>
              <Impressions>30</Impressions>
              <Keyword>Development</Keyword>
       </Ad>
       <Ad>
              <ImageUrl>~/Images/ad3.jpg</ImageUrl>
              <NavigateUrl>https://dotnet.microsoft.com/</NavigateUrl>
              <AlternateText>.NET</AlternateText>
              <Impressions>20</Impressions>
              <Keyword>Technology</Keyword>
       </Ad>
</Advertisements>
```

Images/ad1.jpg, ad2.jpg, ad3.jpg

MISSING: The image files are not in your practicals document (it only says to add them). Add your own images here.

Output:

Expected behaviour (derived by reading the code; not tested in this audit):

On each page load the AdRotator control picks one advertisement from Advertisements.xml, weighted by Impressions (50 : 30 : 20), and shows it at 500 x 200 px with its link and alternate text.

[Insert your output screenshot here - none was supplied, so none has been added]

Practical 12

Company Website converted into a PWA

Aim: Develop a simple company website containing information about product, services, about us, and contact details. Convert it into a PWA using a Web App Manifest and Service Worker.

Project / namespace (as in source): P12CompanyPWA; local URL https://localhost:7258 (both as stated by you - they are not in the practicals document)

Procedure:

[Steps written for this journal from your file list - they are not copied from your source. Adjust the template / framework wording to what you actually used.]

1. Open Visual Studio and click Create a new project, choose the ASP.NET Core template you used for this practical, and click Next.
2. Enter the Project name P12CompanyPWA, choose the Location, click Next, select the installed .NET framework and click Create.
3. In Solution Explorer locate (or create) the wwwroot folder.
4. Right-click wwwroot > Add > New Item and create index.html (the company website with About Us, Products, Services and Contact sections), then enter the code given below.
5. Create manifest.json in wwwroot and enter the Web App Manifest given below (name, short_name, start_url, scope, display, colours and icons).
6. Create service-worker.js in wwwroot and enter the Service Worker code given below, which pre-caches index.html, manifest.json and offline.html.
7. Create offline.html in wwwroot and enter the offline page given below.
8. Add the two icon images icon-192.png (192 x 192) and icon-512.png (512 x 512) to wwwroot, as referenced by the manifest.
9. Make sure Program.cs serves static files (app.UseStaticFiles(), and app.UseDefaultFiles() if you open the site root). Your Program.cs is not in the source document, so check this in your own project.
10. Save all files and run the project with Ctrl+F5. Open https://localhost:7258/index.html (use the port Visual Studio shows) while online so that the Service Worker installs.
11. Press F12 > Application. Check Manifest (name and icons), Service Workers (service-worker.js activated) and Cache Storage (abc-company-v5 containing the cached files).
12. In the Network tab select Offline and reload; the cached home page is displayed, and a page that is not cached shows offline.html.
13. Take screenshots of the website, the Manifest and Service Worker panels, and the offline behaviour. If the browser offers an Install option, you may also capture it.

Source Code:

wwwroot/index.html

Source: your uploaded practicals document - copied exactly (not modified).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1">
    <meta name="theme-color" content="#102a43">
    <link rel="manifest" href="/manifest.json">
    <title>ABC Company</title>
    <style>
        * {
            box-sizing: border-box;
            margin: 0;
            padding: 0
        }

        body {
            font-family: Arial,sans-serif;
            background: #f3f6fa;
            color: #243447;
            line-height: 1.6
        }

        header, footer {
            background: #102a43;
            color: white;
            text-align: center;
            padding: 18px
        }

        nav {
            margin-top: 10px
        }

            nav a {
                color: white;
                text-decoration: none;
                margin: 8px
            }

        .hero {
            background: #166b91;
            color: white;
            text-align: center;
            padding: 45px 15px
        }

            .hero h1 {
                margin-bottom: 12px
            }

        .btn {
            display: inline-block;
            background: white;
            color: #102a43;
            padding: 8px 16px;
```

```
            margin-top: 15px;
            text-decoration: none;
            border-radius: 5px
        }

        section {
            padding: 30px 8%;
            text-align: center
        }

        h2 {
            color: #102a43;
            margin-bottom: 12px
        }

        .cards {
            display: flex;
            flex-wrap: wrap;
            justify-content: center;
            gap: 15px;
            margin-top: 18px
        }

        .card {
            background: white;
            border: 1px solid #ddd;
            border-radius: 8px;
            padding: 20px;
            flex: 1 1 180px;
            max-width: 280px
        }

            .card h3 {
                color: #166b91;
                margin-bottom: 8px
            }

        #contact {
            background: #e5eff6
        }

        footer {
            font-size: 13px
        }
    </style>
</head>
<body>
    <header>
        <h2 style="color:white">ABC COMPANY</h2>
        <nav><a href="#about">About</a><a href="#products">Products</a><a
href="#services">Services</a><a href="#contact">Contact</a></nav>
    </header>
    <main>
        <div class="hero">
            <h1>Innovative Solutions for a Better Future</h1>
            <p>Your trusted partner for quality products and services.</p>
            <a class="btn" href="#services">Explore Services</a>
        </div>
        <section id="about">
            <h2>About Us</h2>
```

```
            <p>We provide reliable business solutions focused on quality,
technology and customer satisfaction.</p>
        </section>
        <section id="products">
            <h2>Our Products</h2>
            <div class="cards">
                <div class="card"><h3>Business Solutions</h3><p>Improve business
operations.</p></div>
                <div class="card"><h3>Digital Products</h3><p>Modern tools for
productivity.</p></div>
                <div class="card"><h3>Smart Systems</h3><p>Technology for business
growth.</p></div>
            </div>
        </section>
        <section id="services">
            <h2>Our Services</h2>
            <div class="cards">
                <div class="card"><h3>Technology</h3><p>Modern technology
solutions.</p></div>
                <div class="card"><h3>Consulting</h3><p>Professional business
guidance.</p></div>
                <div class="card"><h3>Customer Support</h3><p>Reliable customer
assistance.</p></div>
            </div>
        </section>
        <section id="contact">
            <h2>Contact Us</h2>
            <p>Email: info@abccompany.com</p>
            <p>We are ready to help your business grow.</p>
        </section>
    </main>
    <footer>
        <p>&copy; 2026 ABC Company. All rights reserved.</p>
        <p>Progressive Web Application</p>
    </footer>
    <script>
if("serviceWorker" in navigator){navigator.serviceWorker.register("/service-
worker.js");}
    </script>
</body>
</html>
```

wwwroot/manifest.json

Source: your uploaded practicals document - copied exactly (not modified).

```
{
  "name": "ABC Company",
  "short_name": "ABC",
  "start_url": "/index.html",
  "scope": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#123456",
  "description": "ABC Company Progressive Web App",
  "icons": [
    {
      "src": "/icon-192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
```

```
    },
    {
      "src": "/icon-512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    }
  ]
}
```

wwwroot/service-worker.js

Source: your uploaded practicals document - copied exactly (not modified).

```
const CACHE_NAME = "abc-company-v5";

const FILES_TO_CACHE = [
    "/index.html",
    "/manifest.json",
    "/offline.html"
];

self.addEventListener("install", event => {
    event.waitUntil(
        caches.open(CACHE_NAME).then(cache => {
            return cache.addAll(FILES_TO_CACHE);
        })
    );
    self.skipWaiting();
});

self.addEventListener("activate", event => {
    event.waitUntil(
        caches.keys().then(cacheNames => {
            return Promise.all(
                cacheNames
                    .filter(name => name !== CACHE_NAME)
                    .map(name => caches.delete(name))
            );
        })
    );
    self.clients.claim();
});

self.addEventListener("fetch", event => {
    event.respondWith(
        caches.match(event.request).then(cachedResponse => {
            if (cachedResponse) {
                return cachedResponse;
            }

            return fetch(event.request).catch(() => {
                return caches.match("/offline.html");
            });
        })
    );
});
```

wwwroot/offline.html

Source: your uploaded practicals document - copied exactly (not modified).

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>You're Offline</title>
</head>
<body>
    <h1>ABC Company</h1>
    <h2>You are currently offline!</h2>
    <p>Please check your internet connection and try again.</p>
    <a href="/index.html">Return to Home Page</a>
</body>
</html>
```

Program.cs

MISSING: Program.cs for P12CompanyPWA is not in your practicals document. Not invented - please supply it.

wwwroot/icon-192.png, wwwroot/icon-512.png

MISSING: Icon files are referenced by manifest.json but are not in your practicals document.

Output:

Expected behaviour (derived by reading the code; installation and offline use were NOT tested):

The site shows About Us, Our Products, Our Services and Contact Us sections for 'ABC Company'. The Service Worker pre-caches /index.html, /manifest.json and /offline.html and answers requests cache-first; if a request is not cached and the network fails, /offline.html is shown.

[Insert your output screenshot here - none was supplied, so none has been added]